

1. Diferencias Fundamentales: Creacionales vs. Estructurales

La principal distinción radica en el **problema que resuelven**:

Característica	Patrones Creacionales	Patrones Estructurales
Objetivo Principal	Se centran en cómo se crean los objetos. Buscan desacoplar la creación del objeto de su uso, ofreciendo flexibilidad sobre <i>qué</i> objeto se crea, <i>quién</i> lo crea y <i>cuándo</i> .	Se centran en cómo se componen las clases y objetos para formar estructuras más grandes. Buscan simplificar la estructura identificando relaciones simples entre entidades.
Problema que resuelven	Complejidad en la instanciación, dependencia directa de clases concretas, control del ciclo de vida (singleton), o creación de familias de objetos relacionados.	Incompatibilidad de interfaces, necesidad de añadir responsabilidades dinámicamente, o gestión de jerarquías complejas de objetos.
Enfoque	"¿Cómo obtengo la instancia correcta sin acoplarme a su implementación?"	"¿Cómo hago que estas clases trabajen juntas si no fueron diseñadas para ello o para organizar mejor su relación?"
Ejemplos Clave	Singleton, Factory Method, Abstract Factory, Builder, Prototype.	Adapter, Decorator, Facade, Composite, Proxy.

2. Uso en Back-End (Server-Side)

En el back-end, donde la gestión de recursos, la escalabilidad y la lógica de negocio son críticas, estos patrones juegan roles distintos:

Patrones Creacionales en Back-End

- **Singleton:** Muy común para gestionar conexiones a bases de datos, servicios de logging o configuraciones globales de la aplicación. Garantiza que solo haya una instancia compartida en memoria.

- *Ejemplo:* Una clase `DatabaseConnection` que asegura una única pool de conexiones.
- **Factory Method / Abstract Factory:** Útil cuando el sistema debe trabajar con diferentes tipos de almacenamiento o proveedores de servicios (ej. AWS S3 vs. Azure Blob Storage) sin cambiar la lógica de negocio.
 - *Ejemplo:* Una fábrica que devuelve el servicio de notificaciones correcto (Email, SMS, Push) según la configuración del usuario.
- **Builder:** Ideal para construir objetos complejos con muchas opciones opcionales, como consultas SQL complejas o cuerpos de respuestas API extensos.
 - *Ejemplo:* Construcción de un objeto Query paso a paso antes de ejecutarlo contra la BD.

Patrones Estructurales en Back-End

- **Adapter:** Permite integrar librerías de terceros o APIs legacy con tu código moderno.
 - *Ejemplo:* Adaptar una API de pago antigua (XML) para que funcione con tu sistema moderno que espera JSON.
 - **Facade:** Simplifica el acceso a subsistemas complejos. En microservicios, un API Gateway actúa como una fachada.
 - *Ejemplo:* Un servicio `OrderService` que internamente coordina `InventoryService`, `PaymentService` y `ShippingService`, exponiendo solo un método `placeOrder()`.
 - **Proxy:** Fundamental para seguridad, caching y lazy loading.
 - *Ejemplo:* Un proxy que verifica permisos de usuario antes de permitir el acceso a un recurso sensible, o un proxy de caché para respuestas de API frecuentes.
 - **Decorator:** Añadir funcionalidad transversal (cross-cutting concerns) como logging, métricas o transacciones sin modificar la clase original.
 - *Ejemplo:* Envolver un servicio de repositorio para registrar el tiempo de ejecución de cada consulta.
-

3. Uso en Front-End (Client-Side)

En el front-end, la gestión del estado, la renderización de UI y la interacción con APIs son los focos principales.

Patrones Creacionales en Front-End

- **Factory Method:** Útil para crear componentes dinámicos basados en datos.
 - *Ejemplo:* Una función que recibe un tipo de widget ("Chart", "Table", "Map") y devuelve la instancia del componente React/Vue/Angular correspondiente.
- **Singleton:** Aunque menos común explícitamente debido a los módulos ES6, se usa en la gestión de estado global o clientes HTTP.
 - *Ejemplo:* Una instancia única de Axios configurada con interceptores, o el store de Redux/Zustand.
- **Builder:** Menos frecuente directamente, pero presente en librerías de construcción de UI o consultas.
 - *Ejemplo:* Librerías como styled-components o constructores de URLs para APIs.

Patrones Estructurales en Front-End

- **Composite:** Es la base de los árboles de componentes en frameworks modernos (React, Vue, Angular).
 - *Ejemplo:* Un componente List que puede contener ítems simples (ListItem) u otras listas anidadas (NestedList), tratándolos uniformemente.
- **Decorator:** Muy relevante en TypeScript y Angular, o mediante Higher-Order Components (HOCs) en React.
 - *Ejemplo:* Un HOC withAuth que envuelve un componente para proteger rutas, o decoradores de clase en Angular para inyección de dependencias.
- **Adapter:** Conectar modelos de datos de la API con la vista.
 - *Ejemplo:* Transformar la respuesta cruda de una API (con campos en snake_case y fechas ISO) en un objeto de modelo limpio (camelCase, objetos Date) que el componente entiende.

- **Facade:** Simplificar el acceso a APIs complejas del navegador o librerías externas.
 - *Ejemplo:* Una utilidad `StorageService` que unifica `localStorage`, `sessionStorage` y cookies bajo una misma interfaz simple `get()` y `set()`.
- **Proxy:** Utilizado internamente por frameworks reactivos (Vue 3, SolidJS) para detectar cambios en el estado.
 - *Ejemplo:* Vue 3 usa Proxy para observar cambios en los objetos de datos y disparar re-renderizados automáticos.

Resumen Comparativo por Contexto

Contexto	Enfoque Creacional (Back/Front)	Enfoque Estructural (Back/Front)
Back-End	Gestión de ciclos de vida, inyección de dependencias, abstracción de persistencia.	Integración de sistemas, seguridad, caching, organización de microservicios.
Front-End	Renderizado dinámico de componentes, gestión de estado global, clientes API.	Composición de UI, adaptación de datos, reactividad, protección de rutas.

Conclusión

- Usa patrones **Creacionales** cuando te enfrentes a la pregunta: *"¿Cómo creo este objeto de manera flexible, segura o eficiente?"*
- Usa patrones **Estructurales** cuando te enfrentes a la pregunta: *"¿Cómo hago que estos objetos o clases trabajen juntos de forma limpia y mantenible?"*

Ambos tipos son complementarios y esenciales para construir arquitecturas de software robustas, escalables y fáciles de mantener en cualquier capa de la aplicación.

Patrones de Comportamiento. Mientras que los creacionales se ocupan de la *instanciación* y los estructurales de la *composición*, los patrones de comportamiento se centran en la **comunicación** y la **responsabilidad** entre objetos.

Su objetivo principal es definir cómo interactúan los objetos entre sí, cómo distribuyen las tareas y cómo fluye el control en la aplicación, promoviendo un bajo acoplamiento y una alta cohesión.

Características clave

Característica	Descripción
Comunicación	Definen cómo los objetos se comunican entre sí ^[1]
Responsabilidad	Distribuyen responsabilidades entre objetos ^[3]
Acoplamiento	Promueven acoplamiento débil entre componentes ^[1]
Algoritmos	Encapsulan algoritmos y flujos de control ^[2]

1. Diferencias Fundamentales: Comportamiento vs. Los Anteriores

Característica	Patrones Creacionales	Patrones Estructurales	Patrones de Comportamiento
Pregunta Clave	¿Cómo creo el objeto?	¿Cómo estructuro/compongo los objetos?	¿Cómo interactúan/se comunican los objetos?
Foco	Instanciación y ciclo de vida.	Relaciones estáticas (herencia, composición).	Algoritmos, flujo de control y asignación de responsabilidades.
Dinámica	Baja (ocurre al inicio).	Media (estructura fija o semi-fija).	Alta (interacciones en tiempo de ejecución).
Ejemplos Clave	Singleton, Factory, Builder.	Adapter, Decorator, Facade.	Observer, Strategy, Command, Iterator, State.

2. Uso en Back-End (Server-Side)

En el back-end, los patrones de comportamiento son cruciales para manejar la lógica de negocio compleja, la asincronía y la escalabilidad de procesos.

Patrones de Comportamiento Clave en Back-End

- **Strategy (Estrategia):** Permite intercambiar algoritmos en tiempo de ejecución. Es vital para políticas de negocio variables.
 - *Ejemplo:* Un servicio de pagos que tiene diferentes estrategias (CreditCardStrategy, PayPalStrategy, CryptoStrategy). El contexto elige la estrategia según la preferencia del usuario sin cambiar el código principal.
 - *Otro ejemplo:* Algoritmos de descuento (NoDiscount, SeasonalDiscount, VIPDiscount).
- **Observer (Observador):** Fundamental para arquitecturas orientadas a eventos y sistemas reactivos.
 - *Ejemplo:* Cuando un pedido cambia de estado a "Enviado", el sistema notifica automáticamente a múltiples suscriptores: el servicio de email (envía confirmación), el servicio de analíticas (registra el evento) y el servicio de inventario (actualiza stock). Esto desacopla el núcleo de la lógica de pedido de los efectos secundarios.
- **Command (Comando):** Encapsula una solicitud como un objeto. Es esencial para colas de tareas, deshacer/rehacer y transacciones.
 - *Ejemplo:* En un sistema de microservicios, cada acción (crear usuario, procesar pago) se convierte en un objeto Command que se pone en una cola (RabbitMQ/Kafka). Esto permite reintentos, logging centralizado y procesamiento asíncrono.
- **Chain of Responsibility (Cadena de Responsabilidad):** Útil para procesar solicitudes a través de una cadena de manejadores.
 - *Ejemplo:* Middleware de autenticación y autorización. Una solicitud pasa por LoggingMiddleware -> AuthMiddleware -> RateLimitMiddleware -> Controller. Si uno falla, la cadena se rompe.
- **Template Method (Método Plantilla):** Define el esqueleto de un algoritmo, dejando pasos específicos para subclases.

- *Ejemplo:* Un proceso de importación de datos que siempre sigue: LeerArchivo -> ValidarDatos -> Transformar -> GuardarEnBD. Los pasos de validación y transformación varían según el tipo de archivo (CSV, Excel, JSON), pero el flujo general es fijo.
-

3. Uso en Front-End (Client-Side)

En el front-end, la interacción del usuario, la gestión del estado de la UI y la navegación son los dominios principales de estos patrones.

Patrones de Comportamiento Clave en Front-End

- **Observer / Pub-Sub:** La base de la reactividad moderna y la comunicación entre componentes no relacionados.
 - *Ejemplo:* En Redux o Vuex, los componentes se "suscriben" al store. Cuando el estado cambia, los componentes observadores se re-renderizan. También usado en Event Emitters nativos del navegador.
- **Strategy:** Útil para variar el comportamiento de la UI o la validación.
 - *Ejemplo:* Diferentes estrategias de validación de formularios (EmailValidation, PhoneValidation) o estrategias de renderizado (ServerSideRendering vs ClientSideRendering) dependiendo del entorno.
- **State (Estado):** Permite a un objeto alterar su comportamiento cuando su estado interno cambia. Muy útil para máquinas de estado finitas en la UI.
 - *Ejemplo:* Un botón de "Submit" que tiene estados: Idle, Loading, Success, Error. Cada estado define si el botón está deshabilitado, qué icono muestra y qué acción realiza al hacer clic. Librerías como XState implementan este patrón explícitamente.
- **Iterator (Iterador):** Proporciona una forma de acceder secuencialmente a los elementos de un agregado sin exponer su representación subyacente.
 - *Ejemplo:* El uso nativo de for...of en JavaScript sobre Arrays, Maps o Sets. En React, el .map() para renderizar listas es una forma funcional de iterar sobre colecciones de datos para generar componentes.
- **Command:** Útil para gestionar el historial de acciones (Undo/Redo) y atajos de teclado.

- *Ejemplo:* En un editor de texto rico (como Draft.js o Slate.js), cada inserción de texto o formato se guarda como un comando en una pila, permitiendo al usuario deshacer la última acción. También en menús contextuales donde cada ítem ejecuta un comando específico.
- **Mediator (Mediador):** Reduce el caos de comunicaciones directas entre muchos componentes.
 - *Ejemplo:* Un componente padre o un store global actúa como mediador entre componentes hermanos que necesitan compartir datos, evitando el "prop drilling" excesivo o comunicaciones directas complejas.

Resumen Comparativo por Contexto

Contexto	Enfoque de Comportamiento (Back-End)	Enfoque de Comportamiento (Front-End)
Problema Típico	Lógica de negocio condicional, procesamiento asíncrono, integración de servicios.	Interacción del usuario, reactividad de la UI, gestión de estado local/global.
Patrones Estrella	Strategy (políticas), Observer (eventos), Command (colas/tareas).	Observer (reactividad), State (UI states), Command (undo/redo).
Beneficio Principal	Flexibilidad algorítmica y desacoplamiento de procesos.	UX fluida, mantenimiento de la lógica de interacción y claridad en el flujo de datos.

Conclusión Integrada

Para tener una visión completa:

1. **Creacionales:** Te ayudan a **construir** los bloques básicos de tu aplicación (objetos, servicios, componentes) de manera flexible.
2. **Estructurales:** Te ayudan a **ensamblar** esos bloques en estructuras coherentes y adaptables (APIs, árboles de componentes, capas de seguridad).
3. **Comportamiento:** Te ayudan a **orquestrar** la danza entre esos bloques, definiendo quién hace qué, cuándo y cómo se comunican (eventos, estrategias de negocio, flujos de UI).

Un buen arquitecto de software no elige un solo tipo, sino que combina patrones de las tres categorías para resolver problemas complejos. Por ejemplo, podrías usar un **Factory** (Creacional) para crear una estrategia de pago específica (**Strategy** - Comportamiento), que luego es envuelta por un **Proxy** (Estructural) para añadir logging.